

Apunte de Clase: Etapas del Proceso de Compilación

Introducción

La compilación es el proceso mediante el cual un programa escrito en un **lenguaje de alto nivel** (como C, C++, Java) se traduce a un **lenguaje de bajo nivel** (código de máquina o ensamblador) que puede ser ejecutado directamente por una computadora.

1. Etapas principales de la compilación

Tradicionalmente, el proceso se divide en cuatro grandes fases:

1. **Preprocesamiento**
2. **Compilación (como etapa)**
3. **Ensamblaje**
4. **Vinculación (Linkeo)**

1.1 Preprocesamiento

- **Objetivo:** Preparar el código fuente antes de la traducción.
- **Acciones típicas:** Inclusión de archivos (`#include`), expansión de macros (`#define`), eliminación de comentarios.
- **Resultado:** Código fuente "limpio" y expandido.

1.2 Compilación (como etapa)

- Convierte el código preprocesado a **código ensamblador** (`.asm`).
- Es la fase más compleja y se subdivide en varias subetapas (Analizador léxico, sintáctico y semántico).

1.3 Ensamblaje

- Convierte el código ensamblador en un **archivo objeto** (`.obj` o `.o`).
- Contiene código máquina pero aún con referencias simbólicas sin resolver.

1.4 Vinculación (Linkeo)

- **Objetivo:** Resolver las referencias externas (bibliotecas, funciones externas).
- **Resultado:** Archivo ejecutable final.

1.5 Carga y Ejecución

- El sistema operativo carga el ejecutable en memoria y transfiere el control al programa.

2. Subetapas del Análisis y la Síntesis (dentro de la Compilación)

2.1 Análisis Léxico (Scanner)

- **Función:** Convierte el código fuente en una secuencia de **tokens** (etiquetas).
- **Ejemplo:** $X = Y + Z * 30 \rightarrow \langle X, 1 \rangle, \langle = \rangle, \langle Y, 2 \rangle, \langle + \rangle, \langle Z, 3 \rangle, \langle * \rangle, \langle 30 \rangle$
- **Errores que detecta:** Caracteres no permitidos, strings sin cerrar, constantes malformadas.

2.2 Análisis Sintáctico (Parser)

- **Función:** Verifica que los tokens cumplan las reglas gramaticales.
- **Resultado:** Árbol de Sintaxis Abstracta (AST).
- **Errores:** Paréntesis sin cerrar, operadores mal ubicados, asignación a constantes.

2.3 Análisis Semántico

- **Función:** Agrega significado al AST (tipos, declaraciones, alcances).
- **Errores:** Tipos incompatibles, variables no declaradas, violación de acceso (`private`, etc.), argumentos incorrectos en funciones.

3. Lenguaje de Representación Intermedia (IR)

¿Qué es? Un lenguaje entre el código fuente y el código objeto, independiente del lenguaje original y la arquitectura destino.

¿Por qué usarlo? Permite desacoplar análisis de síntesis. Con N lenguajes fuente y M arquitecturas, sin IR se necesitan $N \times M$ compiladores; con IR solo N analizadores + M generadores.

Niveles del IR: Alto (similar al fuente), Medio (sin objetos estructurados), Bajo (cercano al ensamblador).

Ejemplos de IR: LLVM IR, Java Bytecode, C (como IR), ASM.js.

4. Generación de Código Intermedio

- Basada en **tuplas** (operaciones de tres direcciones).
- Se representa mediante un **Grafo de Flujo de Control (CFG)**.

Ejemplo: $X = Y + Z * 30$

IR idealizado:

$R1 = \text{intToFloat}(30)$

$R2 = Z * R1$

$R3 = Y + R2$

$X = R3$

5. Optimización Independiente de la Arquitectura

Conjunto de transformaciones sobre el IR que mejoran el programa sin conocer la arquitectura final.

Nota: Muchas optimizaciones son NP-Hard (no existe un algoritmo que sea eficiente), en la práctica buscamos mejora no optimización perfecta.

Estrategias comunes:

- **Constant Folding:** $30 * 2$ reemplaza directamente 60
- **Simplificación algebraica:** $x * 0$ reemplaza directamente 0
- Eliminación de código inalcanzable / muerto
- Recursión de cola → iteración

6. Generación y Mejora de Código Objetivo

- Convierte IR optimizado en **código ensamblador real** para x86, ARM, etc.
- Aprovecha pipeline, registros, caché, múltiples núcleos.

7. Tabla resumen de optimizaciones

Independientes	"Casi Independientes"	No Independientes
Constant Folding	Cálculo eficiente de direcciones	Selección de instrucciones
Simplificación algebraica	Stack frames simplificados	Asignación de registros
Eliminación código inalcanzable	Asignación estática de frames	Uso de modos de direccionamiento
Eliminación código muerto	Inlining	Balanceo entre CPUs
Recursión de cola	Desenrollado de bucles	Aprovechamiento de caché y pipeline

Compilar no es traducir palabra por palabra, sino entender el significado y luego generar código equivalente.

8. Errores comunes según etapa

Etapa	Tipo de error que detecta
Léxico (Scanner)	Caracteres inválidos, strings mal cerrados
Sintáctico (Parser)	Paréntesis sin cerrar, operadores mal ubicados
Semántico	Tipos incorrectos, variables no declaradas, acceso inválido